

Um Curso de Automação com Práticas Experimentais para Estudantes de Física

Mario Reis

August 20, 2026

Contents

Prática 1: Introdução ao Arduino	4
Prática 2: Acender um LED Externo	7
Prática 3: Piscar um LED (Blink)	10
Prática 4: Controle de LED com Botão	11
Prática 5: Controle de LED com Botão e Buzzer	14
Prática 6: Controle de Brilho de LED com LDR	16
Prática 7: Controle de Buzzer com Sensor Ultrassônico	19
Prática 8: Controle de Servo Motor com Potenciômetro	22
Prática 9: Medindo a Temperatura com Monitor Serial	27
Prática Projeto Final 1: Medindo a aceleração da gravidade g por queda livre	32
Prática Projeto Final 2: Medindo a constante de mola k	34

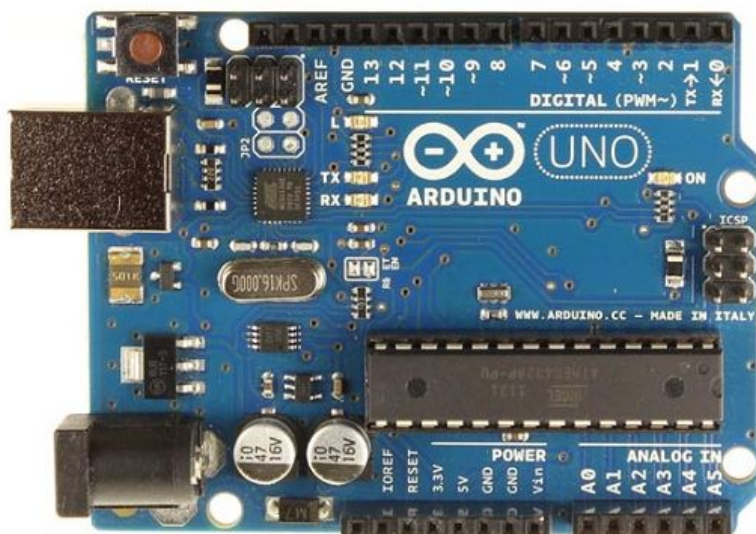


Figure 1: Placa Arduino Uno R3, uma das versões mais populares da plataforma de prototipagem eletrônica. Apresenta microcontrolador ATmega328P, portas digitais e analógicas, conexão USB e outros elementos.

Prática 1: Introdução ao Arduino

O que é o Arduino?

O Arduino é uma plataforma de prototipagem eletrônica de código aberto, baseada em hardware e software fáceis de usar. Ele permite que estudantes e pesquisadores desenvolvam projetos de automação de forma acessível e intuitiva. Veja a Figura [1](#).

Estrutura do Arduino Uno

- **Microcontrolador ATmega328P:** processa os comandos e executa as tarefas. *Localização: centro da placa, componente preto com múltiplos pinos.*
- **Pinos de Entrada e Saída:** conectam sensores e atuadores. *Localização: bordas superiores e inferiores da placa, incluindo portas digitais (D0 a D13) e analógicas (A0 a A5).*
Localização: próximo ao conector de alimentação externa (DC Jack).

- **Conector USB:** permite a comunicação com o computador e alimentação da placa.
Localização: canto superior esquerdo da placa, conector metálico grande.

Neste momento, estes são os componentes mais importantes para identificarmos. Faça o exercício de localizá-los.

Principais Portas e suas Funcionalidades

Portas Digitais

As portas digitais do Arduino operam com dois estados discretos: HIGH=1, que pode ser 5 V ou 3.3 V dependendo do modelo; ou LOW=0, que corresponde a 0 V. Estas portas podem ser configuradas como *entrada* para ler o estado de sensores ou botões; ou *saída*, para controlar LEDs, motores, relés, entre outros dispositivos. Algumas portas digitais possuem suporte a *PWM* (modulação por largura de pulso), permitindo a simulação de sinais analógicos.

Portas Analógicas

As portas analógicas do Arduino são utilizadas para a leitura de sinais variáveis de tensão provenientes de sensores e dispositivos que não operam apenas com valores discretos (0 ou 1), mas sim dentro de uma faixa contínua. O Arduino possui um conversor Analógico-Digital (ADC) de 10 bits, que transforma o valor de tensão lido em um número inteiro dentro de um intervalo de 0 a 1023. Esse intervalo representa os níveis discretizados da tensão medida, onde 0 corresponde a 0 V (ausência de tensão) e 1023 corresponde a 5V (ou 3.3V, dependendo do modelo do Arduino). Isso significa que a resolução da leitura é de aproximadamente 4,88 mV por unidade ($5V/1024$), o que permite detectar pequenas variações no sinal de entrada. Essas portas funcionam apenas no modo entrada e são amplamente utilizadas para a conexão de sensores que fornecem leituras contínuas, como: sensores de temperatura (termistores), de luz (fotorresistores LDR), de umidade e outros.

Tabela Comparativa

Característica	Portas Digitais	Portas Analógicas
Valores possíveis	0 ou 1 (LOW/HIGH)	0 a 1023
Identificação	D0 a D13	A0 a A5
Modo de operação	Entrada e Saída	Apenas Entrada
Exemplo de uso	Botões, LEDs, Relés	Sensores de temperatura, luz,...
Suporte a PWM	Sim, nos pinos com ~	Não

Table 1: Comparação entre portas digitais e analógicas no Arduino

Tinkercad: Simulador para Projetos Arduino

O **Tinkercad** é uma plataforma online que permite simular circuitos eletrônicos sem a necessidade de um hardware físico. Como uma alternativa acessível para quem não tem experiência em programação, o Tinkercad oferece um sistema de criação de código por blocos, facilitando a compreensão inicial. Durante o curso, utilizaremos o Tinkercad para desenvolver nossos primeiros projetos, focando no aprendizado progressivo e na compreensão da lógica de programação sem exigir escrita direta de código.

Arduino Cloud (Web-IDE)

O **Arduino Cloud (Web-IDE)** será a ferramenta utilizada para carregar os programas desenvolvidos no Tinkercad para o hardware real. O fluxo de trabalho seguirá um processo gradativo, no qual os alunos inicialmente construirão os programas por meio de blocos no Tinkercad e, conforme avançam nas práticas, ganharão familiaridade com o código gerado.

O processo ocorrerá da seguinte forma:

1. Criar e simular o circuito no Tinkercad.
2. Copiar o código gerado automaticamente pela interface de blocos.
3. Colar o código no Arduino Cloud (Web-IDE) e carregá-lo no hardware físico.

Ao longo do curso, os alunos irão se familiarizar progressivamente com a escrita de códigos, tornando-se capazes de modificar e criar seus próprios programas.

Prática 2: Acender um LED Externo

Objetivo

Familiarizar-se com a programação por blocos, a simulação no Tinkercad e a interface do Arduino Cloud (Web-IDE). Além disso, aprender a conectar um LED a um Arduino utilizando uma protoboard, compreendendo a importância do resistor para limitar a corrente elétrica e garantir o funcionamento adequado do circuito.

Materiais

- 1 LED (vermelho, verde ou azul)
- 1 Resistor adequado (cálculo explicado abaixo)
- 1 Placa Arduino Uno
- 1 Protoboard
- Jumpers (fios de conexão)

Descrição dos Materiais

Resistor

Para evitar que o LED queime, devemos limitar a corrente elétrica que passa por ele. Considere que a corrente ideal para aumentar a vida útil do LED deve ser inferior a 20 mA. A resistência mínima necessária pode ser calculada utilizando a Lei de Ohm:

$$R = \frac{V_{cc} - V_{LED}}{I}, \quad (1)$$

onde $V_{cc} = 5V$ (saída do Arduino); V_{LED} depende da cor do LED (por exemplo, 2V para vermelho, 3V para azul); e $I = 15$ a 20 mA (corrente recomendada). Por exemplo, para um LED vermelho ($V_{LED} = 2V$) e $I = 15$ mA = 0.015 A, temos $R \approx 200 \Omega$. Portanto, um resistor de 220 Ω é uma boa escolha.

Protoboard

A protoboard é uma matriz de conexões utilizada para montagem de circuitos eletrônicos sem a necessidade de solda. Ela permite testar circuitos de forma rápida e modular. Veja a Figura [2](#)

- As colunas laterais, identificadas com + e –, são barras de alimentação e possuem todos os furos interligados ao longo de sua extensão. Elas são utilizadas para distribuir a tensão e o terra (*GND*) do circuito.
- A área central da protoboard é dividida em duas seções separadas por um canal central. Cada seção contém linhas horizontais de conexões interligadas em grupos de cinco furos: os furos das colunas *a até e* estão conectados entre si, assim como os furos das colunas *f até j*.
- O canal central geralmente é usado para acomodar circuitos integrados (*ICs*), pois ele separa eletricamente as conexões das colunas *a-e* das colunas *f-j*, evitando curto-circuitos indesejados.

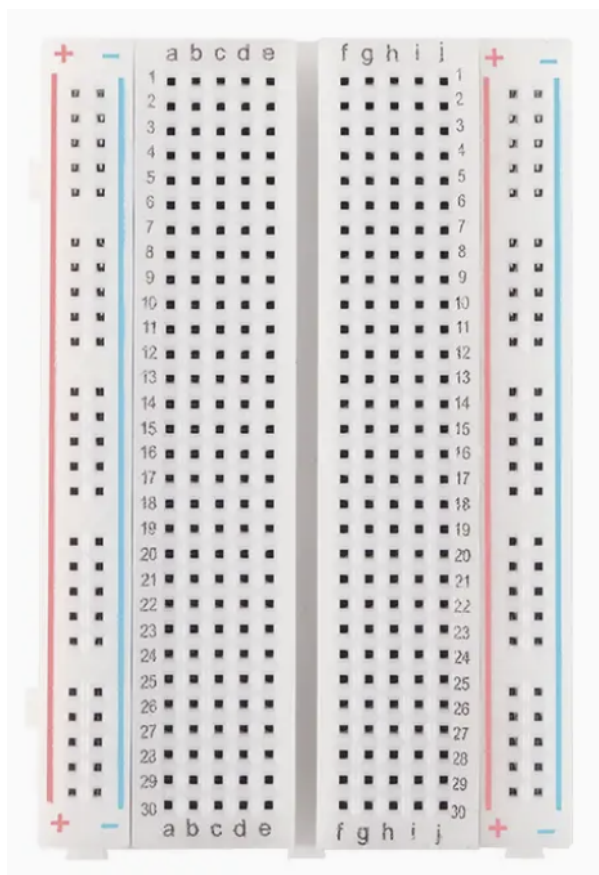


Figure 2: Protoboard padrão com 30 linhas e duas colunas principais de conexões, separadas por um canal central. As barras laterais (marcadas com + e -) são usadas para distribuição de energia e terra.

Prepare o Experimento

1. Criar o circuito no Tinkercad, conectando:
 - O *ânodo* (perna longa) do LED ao pino 13 do Arduino.
 - O *cátodo* (perna curta) ao GND do Arduino, através do resistor.
2. Utilizar a interface de blocos para programar o LED: acender e permanecer ligado.
3. Verificar se a simulação funciona corretamente.

Realize o Experimento

1. Inserir o LED na protoboard, com o *ânodo* (perna longa) conectado a uma linha e o *cátodo* (perna curta) a outra linha distinta.
2. Posicionar um resistor entre o cátodo do LED e a barra de GND da protoboard, garantindo a limitação da corrente. Atenção ao padrão de cores (veja Tabela 2).
3. Utilizar um fio vermelho para conectar o *ânodo* do LED ao pino digital 13 do Arduino, permitindo que o LED seja acionado diretamente pela porta digital.

4. Utilizar um fio preto para conectar a barra de GND da protoboard ao GND do Arduino, garantindo o fechamento do circuito.
5. Revisar todas as conexões, verificando a correta disposição dos fios e componentes, antes de carregar o código no Arduino.
6. Copiar o código gerado no Tinkercad e colar no Arduino Cloud (Web-IDE).
7. Transferir o código para o Arduino, compilar e executar.
8. Verificar se o LED acende corretamente.

Componente	Fio	Símbolo
GND (Terra)	Preto	–
Alimentação Positiva	Vermelho	+

Table 2: Identificação dos fios e símbolos elétricos.

Analise os Resultados

- **Circuito e Componentes:** Papel do resistor na proteção do LED e justificativa do valor escolhido.
- **Simulação e Implementação:** Diferenças entre a simulação no Tinkercad e a montagem física. Dificuldades encontradas na montagem e como foram resolvidas.
- **Automação e Código:** Como o código permite controlar o LED. Possibilidades de modificação do código para outras funcionalidades do LED.

Prática 3: Piscar um LED (Blink)

Objetivo

Familiarizar-se com a programação por blocos, a simulação no Tinkercad e a interface do Arduino Cloud (Web-IDE). Além disso, aprender a controlar um LED utilizando um pino digital do Arduino, alternando entre os estados ligado e desligado.

Materiais

- 1 LED
- 1 Resistor (maior que) $220\ \Omega$
- 1 Placa Arduino Uno
- 1 Protoboard
- Jumpers (fios de conexão)

Prepare o Experimento

1. A montagem do circuito no Tinkercad é a mesma da Prática 1.
2. Programar um pisca-pisca (Blink) utilizando blocos de programação.
3. Definir tempos distintos para o LED ligado e desligado (exemplo: 1 segundo ligado, 500 ms desligado).
4. Testar a simulação e verificar se o LED pisca corretamente.

Realize o Experimento

1. A montagem do circuito na Protoboard é a mesma da Prática 1.
2. Copiar o código gerado no Tinkercad e colá-lo no Arduino Cloud (Web-IDE).
3. Transferir o código para o Arduino e compilar.
4. Executar o código e verificar se o LED pisca conforme programado.

Analise os Resultados

- **Simulação e Implementação:** Verificar se há diferença perceptível entre os tempos programados e os tempos observados no simulador (Tinkercad) e no hardware real.
- **Automação e Código:** Comentar possíveis variações no tempo de pisca e discutir ideias de controle baseadas no uso do pisca-pisca, como sinalização, alertas ou sequências com múltiplos LEDs.

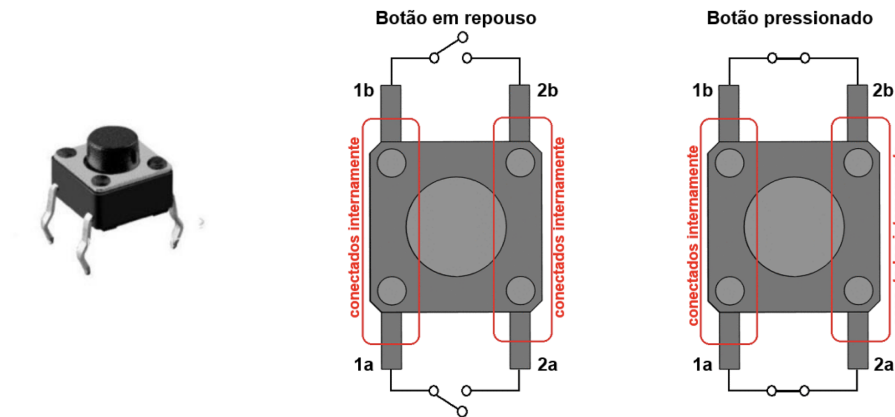


Figure 3: Esquema do funcionamento de um botão de quatro terminais. Os terminais da mesma lateral (1a e 1b, 2a e 2b) estão sempre conectados internamente. Quando o botão está em repouso, não há conexão entre os terminais opostos (1a e 2a). Quando o botão é pressionado, ocorre um curto-circuito entre 1a e 2a, permitindo a passagem de corrente.

Prática 4: Controle de LED com Botão

Objetivo

Familiarizar-se com a entrada digital do Arduino, aprendendo a ler o estado de um botão e controlar um LED com base nessa entrada. Além disso, compreender o funcionamento de resistores de *pull-down* para evitar leituras indesejadas no pino de entrada digital.

Materiais

- 1 LED
- 1 Resistor de $220\ \Omega$ (para o LED)
- 1 Resistor de $10k\Omega$ (pull-down para o botão)
- 1 Botão (switch)
- 1 Placa Arduino Uno
- 1 Protoboard
- Jumpers (fios de conexão)

Descrição dos Materiais

Botão

O botão possui quatro terminais, onde os pinos da mesma lateral estão sempre conectados internamente (1a com 1b e 2a com 2b). Quando o botão não está pressionado, os terminais 1a e 2a (assim como 1b e 2b) permanecem desconectados. Quando o botão é pressionado, ocorre um curto-circuito interno entre 1a e 2a (e também entre 1b e 2b), fechando o circuito e permitindo a passagem de corrente. Veja Figura [3](#).

Prepare o Experimento

1. Criar o circuito no Tinkercad, conectando:
 - O LED ao pino digital 13 do Arduino, assim como foi feito na prática anterior.
 - O pino 1b do botão ao pino digital 2 do Arduino.
 - O resistor de *pull-down* (10 k Ω) entre o pino 1a do botão e o GND do Arduino.
 - O pino 2a do botão ao pino de 5 V do Arduino.
2. Entenda a conexão do botão:
 - Os terminais 1a e 1b devem ser conectados ao pino digital 2 do Arduino, pois será a entrada digital monitorada pelo microcontrolador.
 - O terminal 2a deve ser conectado de modo a alternar entre GND e 5V, determinando se o pino 2 do Arduino recebe um nível LOW (0V) ou HIGH (5V).
 - Botão solto:
 - Os terminais 1a e 2a estão abertos, ou seja, não há conexão entre eles.
 - Como 1a e 1b estão conectados internamente, o pino digital 2 do Arduino fica diretamente ligado ao GND através de um resistor de 10k Ω (*pull-down*).
 - Isso garante que o pino digital 2 do Arduino permaneça em nível LOW (0V), evitando leituras flutuantes ou instáveis.
 - Botão pressionado:
 - Os terminais 1a e 2a se conectam internamente, fechando o circuito.
 - O terminal 2a está conectado à porta de 5V do Arduino, permitindo que a corrente flua diretamente até o pino digital 2 do Arduino.
 - Agora, o pino digital 2 do Arduino recebe um nível HIGH (5V), indicando que o botão foi pressionado.
 - Essa transição de LOW para HIGH será detectada pelo Arduino e poderá ser usada no código para ativar o LED.
3. Programar o código para ligar o LED quando o botão for pressionado e desligá-lo quando o botão for solto.
4. Testar a simulação no Tinkercad e garantir que o LED acenda e apague conforme o botão é pressionado e liberado.

Realize o Experimento

1. Montar o circuito na protoboard, seguindo o esquema do Tinkercad.
2. Copiar o código gerado no Tinkercad e colá-lo no Arduino Cloud (Web-IDE).
3. Transferir o código para o Arduino e compilar.
4. Executar o código e verificar se o LED acende quando o botão é pressionado e apaga quando é liberado.
5. Testar diferentes resistores de *pull-down* e observar o impacto nas leituras do botão.

Analise os Resultados

- **Leitura Digital e Comportamento do Botão:** Observar como o Arduino interpreta os sinais enviados pelo botão. Verificar se há falhas ou atrasos na resposta ao pressionar ou soltar o botão.
- **Simulação vs. Montagem Real:** Analisar se o comportamento observado na simulação (Tinkercad) corresponde ao da montagem física. Destacar eventuais discrepâncias.
- **Resistores de Pull-down:** Comentar o papel do resistor de $10k\Omega$ na estabilidade da leitura digital. Refletir sobre os efeitos de sua ausência ou substituição por outro valor.
- **Código e Automação:** Avaliar como o código responde ao acionamento do botão. Discutir possíveis melhorias ou novas aplicações, como acionar mais de um LED, controlar tempo de acionamento, entre outros.

Prática 5: Controle de LED com Botão e Buzzer

Objetivo

Expandir o conceito de entrada digital do Arduino, combinando o controle de um LED e de um *buzzer* a partir da leitura do estado de um botão. Utilizar o *buzzer* para emitir um som quando o botão for pressionado, reforçando o acionamento do LED com um sinal sonoro.

Materiais

- 1 LED
- 1 Resistor de $220\ \Omega$ (para o LED)
- 1 Resistor de $10\ k\Omega$ (pull-down para o botão)
- 1 Botão (switch)
- 1 Buzzer (ativo ou passivo)
- 1 Placa Arduino Uno
- 1 Protoboard
- Jumpers (fios de conexão)

Descrição dos Materiais

Buzzer

O *buzzer* é um componente eletrônico que emite um som quando alimentado por um sinal elétrico. Existem dois tipos principais de *buzzer*:

- *Buzzer Ativo*: Possui um circuito interno que gera o som automaticamente quando recebe uma tensão de alimentação (normalmente 5V). Basta ligá-lo ao Arduino para que emita um som contínuo.
- *Buzzer Passivo*: Não possui um circuito interno de geração de som e precisa de um sinal PWM do Arduino para emitir diferentes tons.

Nesta prática, utilizaremos um *buzzer ativo*, que será acionado quando o botão for pressionado, emitindo um som para complementar o acionamento do LED. O buzzer será conectado a um dos pinos digitais do Arduino e ativado por um comando no código.

Prepare o Experimento

1. Mantenha todas as ligações da prática anterior e adicione o buzzer.
2. Conecte o buzzer ao Arduino:
 - O polo positivo (+) do buzzer deve ser conectado ao pino digital 8 do Arduino.
 - O polo negativo (-) do buzzer deve ser conectado ao GND da placa.

3. Atualize o código por blocos no Tinkercad.
4. Teste a simulação no Tinkercad e garanta que o LED acende e apaga conforme o botão é pressionado e liberado. Em simultâneo, o buzzer deve emitir um som.

Realize o Experimento

1. Montar o circuito na protoboard, seguindo o esquema do Tinkercad. Se necessário, reveja a prática anterior.
2. Copiar o código gerado no Tinkercad e colá-lo no Arduino Cloud (Web-IDE).
3. Transferir o código para o Arduino e compilar.
4. Executar o código e verificar se:
 - O LED acende quando o botão é pressionado e apaga quando é liberado.
 - O buzzer emite um som enquanto o botão estiver pressionado.
5. Gerar diferentes frequências sonoras. Altere esta condição no Arduino Cloud (Web-IDE).

Analise os Resultados

- **Resposta Auditiva e Visual:** Comentar sobre a utilidade da combinação som/luz como forma de reforçar ações em sistemas automatizados.
- **Extensões e Variações:** Explorar possibilidades de modificação, como ativar o buzzer com diferentes padrões sonoros (frequências ou duração).
- **Criação de Aplicações Práticas:** Desenvolver uma proposta de sistema que utilize LED(s) e buzzer para alguma função específica, como:
 - Um alarme simples para simular uma campainha ou sistema de aviso.
 - Um semáforo básico com tempos de acionamento e alerta sonoro.
 - Um contador manual com botão e sinalização sonora/visual.
- **Prototipagem Livre:** Com base nos componentes estudados até aqui, elaborar um pequeno projeto próprio (com botão, LED e buzzer) que execute alguma função de interesse do aluno/grupo. Descreva o projeto no relatório.

Prática 6: Controle de Brilho de LED com LDR

Objetivo

Nesta prática, os alunos irão desenvolver um sistema de controle automático de brilho de um LED com base na luminosidade do ambiente, utilizando um sensor LDR em conjunto com a técnica de modulação por largura de pulso (PWM). O objetivo principal é proporcionar uma compreensão integrada de conceitos fundamentais de eletrônica e automação, como leitura analógica de sensores, construção de divisores de tensão e controle proporcional de atuadores.

Materiais

- 1 x Placa Arduino Uno
- 1 x LDR (Light Dependent Resistor)
- 1 x Resistor de $10k\Omega$ (para o divisor de tensão)
- 1 x LED (qualquer cor)
- 1 x Resistor de 220Ω (para limitar a corrente do LED)
- 1 x Protoboard
- Jumpers (fios de conexão)

Descrição dos Materiais

LDR (Light Dependent Resistor)

O LDR é um sensor que varia sua resistência conforme a quantidade de luz incidente. Em ambientes escuros, sua resistência é alta (acima de $1M\Omega$). Em ambientes claros, sua resistência diminui (cerca de $1k\Omega$ ou menos). Essa variação de resistência pode ser usada para medir a luz ambiente, mas o Arduino não pode ler resistência diretamente. Por isso, usamos um divisor de tensão.

Divisor de Tensão

O divisor de tensão permite transformar a resistência do LDR em tensão variável para que o Arduino possa medi-la. Ele é formado pelo LDR e um resistor fixo de $10k\Omega$, conectados da seguinte forma (veja Figura 4):

- O LDR conecta-se entre 5V e o ponto central do divisor.
- O resistor de $10k\Omega$ conecta-se entre o ponto central do divisor e o GND.
- O ponto central do divisor (entre LDR e resistor de $10k\Omega$) conecta-se ao pino analógico A0 do Arduino.

A tensão resultante V_{out} no ponto central segue a fórmula (verifique!):

$$V_{out} = V_{cc} \times \frac{R_{10k\Omega}}{R_{LDR} + R_{10k\Omega}} \quad (2)$$

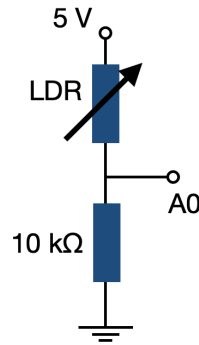


Figure 4: Esquema do divisor de tensão utilizando um LDR e um resistor de 10kΩ. A tensão medida no ponto intermediário (pino A0) varia de acordo com a luminosidade incidente no LDR.

Assim, se houver muita luz, R_{LDR} diminui e V_{out} aumenta; se estiver escuro, R_{LDR} aumenta e V_{out} diminui. O Arduino lê o V_{out} e converte para um valor entre 0 e 1023 no pino analógico A0.

PWM (Pulse Width Modulation)

O PWM é um método usado pelo Arduino para simular uma saída analógica em um pino digital. No caso do LED, o Arduino gera um sinal que liga e desliga rapidamente. A frequência do PWM é tão alta que o olho humano não percebe os pulsos, mas sim um brilho intermediário. O controle do brilho é feito ajustando o tempo que o LED fica ligado (duty cycle). O Arduino aceita valores PWM de 0 a 255, sendo 0 (LED apagado) e 255 (LED no brilho máximo). Para valores intermediários de PWM, o LED terá um brilho proporcional. Nesta Prática, vamos mapear os valores do LDR (0-1023) para o PWM (0-255), garantindo que o brilho do LED responda à luz ambiente.

Prepare o Experimento

1. Montagem do simulador

- LDR: Um terminal no 5V do Arduino; outro terminal no pino A0 e também ao resistor de 10kΩ.
- Resistor de 10kΩ: Um lado no pino A0, outro lado no GND.
- LED: O cátodo (perna curta) vai para o GND. O ânodo (perna longa) vai ao resistor de 220Ω.
- Resistor de 220Ω: Um lado no ânodo do LED. O outro lado no pino D9 (saída PWM do Arduino).

2. Montagem do Software por Blocos

- Selecionar um bloco de leitura analógica e configurar a leitura para o pino A0.
- Escolher um bloco de mapeamento de valor e configurar a conversão do intervalo 0-1023 (entrada analógica) para 0-255 (saída PWM).

- Escolher um bloco de escrita analógica (PWM) e configurar para o pino 9 (onde o LED está conectado). Conectar esse bloco ao bloco de mapeamento.
- Envolver todos os blocos dentro de um bloco de repetição infinita (loop infinito), para manter o funcionamento contínuo.
- Testar a simulação. Se necessário, adicionar um pequeno atraso (10 ms).

Realize o Experimento

1. Montar o circuito na protoboard conforme o esquema virtual do Tinkercad.
2. Copiar o código gerado no Tinkercad e colá-lo no Arduino Cloud (Web-IDE).
3. Transferir o código para o Arduino e compilar.

Analise os Resultados

- **Análise do Comportamento:** Compare o brilho do LED em diferentes condições de luminosidade. O comportamento está coerente com o esperado? Há algum atraso perceptível na resposta? O sistema responde suavemente à variação de luz ou apresenta saltos bruscos?
- **Criação de Aplicações Práticas:** Proponha uma aplicação prática que utilize o LDR e o LED, como:
 - Um sistema de iluminação automática que acende à noite.
 - Um indicador visual de luminosidade para plantas sensíveis à luz.
 - Um alarme que detecta variação brusca de luz (como uma lanterna em um ambiente escuro).
- **Prototipagem:** Com base nos componentes estudados até agora (botão, buzzer, LED, LDR), desenvolva um pequeno projeto funcional. Por exemplo:
 - Um alarme visual-sonoro acionado por alteração na luminosidade.
 - Um sistema de resposta dupla: se está escuro, acende LED; se está claro demais, aciona um som (buzzer).

Descreva o funcionamento do seu projeto no relatório.



Figure 5: Sensor ultrassônico HC-SR04 utilizado para medição de distância. Possui quatro pinos: VCC (alimentação de 5V), Trig (gatilho para envio do pulso ultrassônico), Echo (recebimento do pulso refletido) e GND (terra).

Prática 7: Controle de Buzzer com Sensor Ultrassônico

Objetivo

Esta prática tem como objetivo ensinar o uso do sensor ultrassônico HC-SR04 para medir distâncias. Para este fim, iremos controlar um buzzer passivo, emitindo diferentes frequências sonoras conforme a aproximação de um objeto. Os alunos irão aprender a integrar o sensor ultrassônico com atuadores sonoros no Arduino, abordando conceitos como entrada e saída digitais, tempo de resposta e uso de condicionais no código.

Materiais

- 1 Placa Arduino Uno
- 1 Sensor Ultrassônico HC-SR04
- 1 Buzzer passivo
- 1 Protoboard
- Jumpers (fios de conexão)

Descrição dos Materiais

Sensor Ultrassônico HC-SR04

O sensor HC-SR04 mede distâncias emitindo um pulso ultrassônico e calculando o tempo até o retorno do eco após a reflexão em um obstáculo. Suas conexões são (veja Figura 5):

- Trig (Trigger): envia o pulso ultrassônico.
- Echo: recebe o pulso refletido e retorna o tempo para o cálculo da distância.
- VCC e GND: alimentação do sensor (5V).

A distância é calculada utilizando a velocidade do som no ar (≈ 343 m/s a 25°C).

Prepare o Experimento

1. O código do Arduino deverá medir a distância e ativar o buzzer conforme a proximidade:
 - Quando algum objeto estiver a mais de 10 cm do buzzer, este toca uma nota C5 (523 Hz), contínua.
 - Quando algum objeto estiver entre 6 cm e 10 cm do buzzer, este toca uma nota D4 (294 Hz), pulsada.
 - Quando algum objeto estiver a menos de 6 cm do buzzer, este toca uma nota E3 (165 Hz), pulsada de maior frequência.
2. Conexões:
 - O pino Trig do sensor HC-SR04 deve ser conectado ao pino digital 9 do Arduino.
 - O pino Echo deve ser conectado ao pino digital 10 do Arduino.
 - O pino VCC deve ser ligado a 5V e o GND ao GND do Arduino.
 - O buzzer passivo deve ter um terminal conectado ao pino digital 6 do Arduino e o outro ao GND.
3. Elaboração do código por blocos:
 - Selecionar um bloco de leitura do sensor ultrassônico e configurar o pino 9 para envio do sinal (Trig) e o pino 10 para recebimento do sinal (Echo). Criar uma variável chamada *distância* para armazenar o valor fornecido pelo bloco do sensor.
 - Criar bloco condicional (*if*) para acionar o buzzer com base na distância:
 - Se a distância for maior ou igual a 10 cm, definir a saída do buzzer com uma nota C5 (523 Hz), contínua.
 - Se a distância estiver entre 6 cm e 10 cm, alterar a frequência do buzzer para uma nota D4 (294 Hz), pulsada.
 - Se a distância for menor ou igual a 6 cm, aumentar a frequência para uma nota E3 (165 Hz), pulsada de maior frequência.
 - Envolver todos os blocos dentro de um bloco de repetição infinita para manter a execução contínua.
 - Testar a simulação no Tinkercad e, se necessário, adicionar um pequeno atraso (10 ms) para evitar leituras instáveis.

Realize o Experimento

1. Montar o circuito na protoboard, seguindo o esquema criado no Tinkercad.
2. Copiar o código gerado no Tinkercad e colar no Arduino Cloud (Web-IDE).
3. Transferir o código para o Arduino e compilar.
4. Testar se o buzzer responde corretamente aos diferentes níveis de aproximação.
5. Verificar, com uma régua, se as distâncias do código, estão corretas.

Analise os Resultados

- **Análise do Comportamento:** Avalie como o buzzer responde à aproximação de objetos. O som emitido varia conforme esperado para cada faixa de distância? Há algum atraso perceptível entre a movimentação do objeto e a mudança na frequência do buzzer? A resposta do sistema é suave ou apresenta falhas?
- **Criação de Aplicações Práticas:** Proponha uma aplicação prática utilizando o sensor ultrassônico e o buzzer, como:
 - Um sistema de alerta de proximidade para estacionamentos (sensor de ré).
 - Um alarme de segurança que detecta a aproximação de pessoas em áreas restritas.
 - Um contador de objetos que atravessam uma linha de detecção.
- **Prototipagem:** Com base nos componentes estudados até agora (botão, buzzer, LED, LDR, sensor ultrassônico), desenvolva um pequeno projeto funcional. Por exemplo:
 - Um sistema de alarme que dispara sons e acende LEDs conforme a distância detectada.
 - Um dispositivo interativo que muda sons e luzes conforme a movimentação de uma pessoa.

Descreva o funcionamento do seu projeto no relatório.

Prática 8: Controle de Servo Motor com Potenciômetro

Objetivo

Nesta prática, vamos controlar a posição de um servo motor utilizando um potenciômetro. Através da leitura do valor analógico do potenciômetro pelo Arduino, será possível converter essa informação em um ângulo de rotação para o servo. Essa atividade permite compreender, de forma prática, os conceitos de leitura analógica, sinal PWM (modulação por largura de pulso) e controle de atuadores, fundamentais em sistemas de automação e robótica.

Materiais

- 1 x Placa Arduino Uno
- 1 x Shield de prototipagem para Arduino
- 1 x Servo motor (SG90 ou similar)
- 1 x Potenciômetro de 10k Ω
- 1 x Protoboard
- Jumpers (fios de conexão)
- Fonte de alimentação externa (opcional)

Descrição dos Materiais

Arduino Sensor Shield V5.0

O *Arduino Sensor Shield V5.0* é uma placa de expansão que facilita a conexão de sensores, módulos e atuadores ao Arduino Uno, organizando os pinos de forma mais acessível e segura. Veja a Figura [6](#). Em vez de utilizar fios diretamente na placa Arduino, a shield apresenta conectores com confortável separação entre os terminais, alimentação (VCC) e terra (GND), o que reduz erros de conexão e melhora a montagem do circuito. Cada pino digital e analógico do Arduino é espelhado na shield com um conector de três vias (Sinal, VCC e GND), permitindo a conexão direta de sensores e componentes. Além disso, a placa oferece interfaces dedicadas para módulos como LCDs, Bluetooth, comunicação I²C, sensores ultrassônicos, módulos seriais, entre outros. Isso torna a integração com diversos periféricos muito mais simples. Outro recurso importante é a presença de um bloco de terminais de alimentação, que permite conectar uma fonte externa de 5V ao circuito, útil quando o consumo de corrente ultrapassa o que a porta USB do computador pode fornecer com segurança. Nesta prática, utilizaremos a shield para organizar a conexão do potenciômetro e do servo motor aos respectivos pinos do Arduino. Com isso, a montagem torna-se mais robusta, acessível e didática, facilitando a visualização e o manuseio dos componentes durante os testes.

Potenciômetro

O potenciômetro é um componente eletrônico que funciona como um resistor variável de três terminais. Ele é composto por uma faixa resistiva e um cursor deslizante (também chamado de *wiper*) que se move conforme o eixo do potenciômetro é girado.

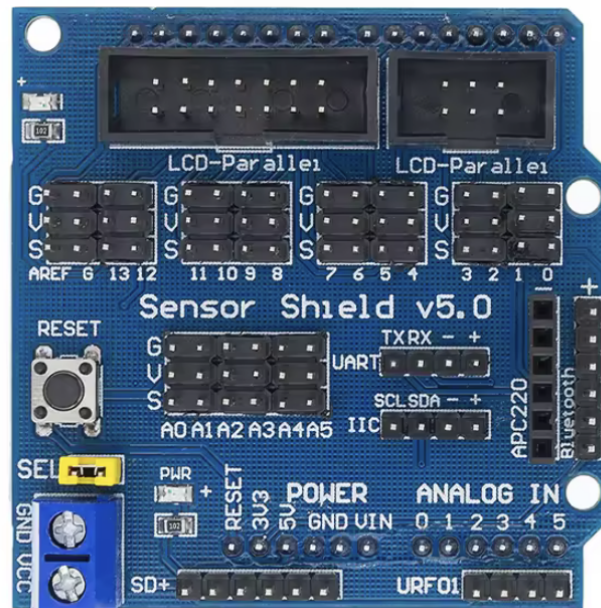


Figure 6: Arduino Sensor Shield V5.0. A placa organiza os pinos digitais e analógicos do Arduino em conectores de três vias (Sinal, VCC e GND), e inclui interfaces dedicadas para comunicação I²C, módulos Bluetooth, sensores ultrassônicos, LCDs e alimentação externa.

- *Terminais laterais (fixos)*: são conectados às extremidades da faixa resistiva. Em geral, um deles é ligado ao terminal de 5V (alta tensão) e o outro ao (GND) (terra, ou 0V).
- *Terminal central (cursor)*: é ligado ao ponto móvel da resistência e fornece uma tensão de saída que varia continuamente entre 0V e 5V, conforme a posição do eixo.

Ao girar o potenciômetro, o cursor se desloca sobre a faixa resistiva, alterando a proporção das resistências entre os dois lados e modificando a tensão de saída. Isso permite que o componente funcione como um *divisor de tensão*. Quando essa tensão é conectada ao pino analógico A0 do Arduino, o microcontrolador interpreta o valor como um número entre 0 e 1023, correspondendo ao intervalo de 0V a 5V. Nesta prática, esse valor será utilizado para controlar o ângulo de um servo motor, estabelecendo uma interface simples e eficaz entre uma entrada analógica (posição do potenciômetro) e uma saída de movimento (posição do servo).

Servo motor SG90

O SG90 é um micro servo motor amplamente utilizado em aplicações educacionais, robótica e sistemas de automação de pequeno porte. Ele é compacto, leve e opera com tensões entre 4.8V e 6V. Seu eixo é capaz de se mover dentro de um intervalo limitado, entre 0° e 180°, de forma precisa e controlada. O controle do SG90 é feito por meio de um sinal PWM (modulação por largura de pulso), no qual a *largura do pulso determina o ângulo do eixo*. Por exemplo:

- Pulso de 1 ms → posição próxima de 0°;
- Pulso de 1,5 ms → posição próxima de 90°;



Figure 7: Servo motor modelo SG90, acompanhado de braços plásticos de controle e parafusos de fixação. O conector de três vias traz os fios de sinal (laranja), alimentação (vermelho) e terra (marrom ou preto).

- Pulso de 2 ms $\rightarrow$ posição próxima de 180° .

Ao girar o eixo do potenciômetro, a tensão na entrada analógica do Arduino (A0) varia de 0V a 5V. O Arduino converte essa tensão em um valor numérico entre 0 e 1023, e este número é mapeado pelo código para um ângulo entre 0° e 180° . Internamente, o SG90 possui um motor DC, um sistema de engrenagens redutoras, um circuito de controle e um potenciômetro interno para realimentação da posição, permitindo manter o eixo na posição desejada mesmo sob pequenas cargas. Esse tipo de servo é ideal para projetos didáticos, braços robóticos, sistemas de orientação, mecanismos de abertura automática e outras aplicações que exigem controle angular simples e confiável.

Prepare o Experimento

1. Monte o simulador no Tinkercad:

- Adicione uma placa Arduino Uno ao projeto.
- Insira um potenciômetro (10 k Ω) no circuito. Conecte: (i) terminal esquerdo ao pino (GND) do Arduino, (ii) terminal direito ao pino 5V do Arduino, e (iii) terminal central ao pino A0 do Arduino.
- Adicione um servo motor (SG90). Conecte: (i) fio vermelho (alimentação) ao pino 5V do Arduino, (ii) fio marrom ou preto (terra) ao pino (GND), e (iii) fio laranja (sinal) ao pino D9 do Arduino.

2. Elaboração do código por blocos:

- Crie duas variáveis: *potenciometro* e *angulo*.
- Defina a variável *potenciometro* como leitura analógica de A0. Defina a variável *angulo* como mapeamento da variável *potenciometro* entre 0° e 180° .
- Defina o servo no pino 9 para angulo que está na variável *angulo*.

- Insira um pequeno atraso com o bloco: **esperar 100 milissegundos**.
- Envolver todos os blocos anteriores dentro de um loop ‘para sempre’, garantindo a execução contínua da leitura e controle.

Realize o Experimento

1. Encaixar a shield de prototipagem sobre a placa Arduino Uno. Embora ela não esteja presente no simulador do Tinkercad, será utilizada na montagem física para facilitar e organizar as conexões dos componentes.
2. Realizar as conexões do potenciômetro e do servo motor conforme indicado no simulador.
3. Copiar o código gerado no Tinkercad (modo Texto) e colar no Arduino Cloud (Web-IDE).
4. Verificar se todas as variáveis foram definidas corretamente como `int` ou `float`, conforme necessário.
5. Compilar e transferir o código para a placa Arduino.
6. Girar o potenciômetro e observar o movimento do servo motor, verificando se o eixo responde suavemente às variações do sinal analógico.
7. Caso o movimento esteja irregular, verificar as conexões, a alimentação do servo e a coerência do mapeamento do ângulo.

Nota: o pino de 5V do Arduino Uno tem sido, ao longo das práticas, alimentado pela porta USB, que fornece uma corrente limitada - geralmente até 500 mA. Parte dessa corrente é consumida pelo próprio Arduino e outros componentes conectados à placa. O servo motor SG90 pode exigir correntes na faixa de 150 mA a 250 mA durante movimento, e picos acima de 500 mA em certas situações (como travamento mecânico ou carga acoplada). Caso o servo apresente comportamento irregular (como tremores, travamentos ou resets do Arduino), recomenda-se o uso de uma fonte de alimentação externa (por exemplo, 4 pilhas AA ou uma fonte regulada de 5V com capacidade de pelo menos 1A).

Análise os Resultados

- **Análise do Funcionamento:** Descreva como o sistema respondeu ao movimento do potenciômetro. O movimento do servo foi suave e proporcional? Houve algum comportamento inesperado?
- **Alimentação e Estabilidade:** Comente se houve necessidade de utilizar alimentação externa. O servo apresentou travamentos, tremores ou resets do Arduino durante os testes?
- **Mapeamento da Leitura:** Avalie a correspondência entre a rotação do potenciômetro e o ângulo final do servo. O intervalo de rotação (0° a 180°) foi respeitado?

- **Extensões e Variações:** Proponha possíveis modificações ou ampliações do sistema, como:
 - Controle simultâneo de dois servos com dois potenciômetros.
 - Substituir o potenciômetro por outro sensor analógico (ex.: LDR, NTC).
- **Prototipagem Livre:** Com base nos componentes estudados até aqui (LEDs, buzzer, LDR, NTC, potenciômetro, servo motor), desenvolva um pequeno projeto próprio. Exemplos:
 - Sistema de abertura com controle manual (potenciômetro).
 - Braço robótico simples com controle de posição.
 - Simulador de limpador de para-brisa com controle de velocidade via potenciômetro.

Descreva o projeto no relatório.

Prática 9: Medindo a Temperatura com Monitor Serial

Objetivo

Nesta prática, vamos medir a temperatura utilizando um sensor NTC 10k, explorando sua variação de resistência com a temperatura. Utilizaremos um divisor de tensão, semelhante ao do experimento com o LDR, para converter essa variação de tensão em um sinal analógico lido pelo Arduino. O valor será processado e exibido no monitor serial, permitindo a leitura direta da temperatura, no monitor do computador.

Materiais

- 1 x Placa Arduino Uno
- 1 x Sensor NTC 10k (termistor)
- 1 x Resistor de $10k\Omega$ (para o divisor de tensão)
- 1 x Protoboard
- Jumpers (fios de conexão)
- Multímetro
- Água com gelo e água fervente (10 ml)
- Termômetro de laboratório (para calibração do sensor).

Descrição dos Materiais

Sensor NTC 10k (Termistor de Coeficiente Negativo de Temperatura)

O termistor NTC (*Negative Temperature Coefficient*) é um sensor de temperatura cuja resistência elétrica diminui com o aumento da temperatura. Esse comportamento ocorre devido à natureza semicondutora do material, no qual o aumento da temperatura aumenta a densidade de portadores de carga, reduzindo a resistência elétrica do componente. A relação entre a resistência R_{NTC} do termistor (em Ohms) e a temperatura absoluta T é dada por:

$$R_{NTC} = R_0 \exp \left\{ B \left(\frac{1}{T} - \frac{1}{T_0} \right) \right\}, \quad (3)$$

onde R_0 é a resistência do termistor em uma temperatura de referência T_0 (tipicamente $T_0 = 298$ K) e B é o coeficiente térmico do termistor (um parâmetro característico do material, usualmente entre 3000 K e 4500 K). A equação acima pode ser reescrita como:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \ln \left(\frac{R_{NTC}}{R_0} \right). \quad (4)$$

Como o Arduino não mede resistência diretamente, empregamos um divisor de tensão constituído pelo sensor NTC e por um resistor fixo $R_{10k\Omega}$. A variação de resistência do termistor é, assim, convertida em uma tensão analógica V_{out} aplicada ao pino A0. Para

obter a expressão de V_{out} , considere que R_{NTC} (entre V_{cc} e o nó de leitura) está em série com $R_{10k\Omega}$ (entre o nó de leitura e o GND). A mesma corrente I percorre ambos:

$$I = \frac{V_{cc}}{R_{NTC} + R_{10k\Omega}}. \quad (5)$$

A queda de tensão em $R_{10k\Omega}$, que é justamente V_{out} , vale:

$$V_{out} = I R_{10k\Omega}. \quad (6)$$

Levando a Equação 5 na Equação 6 obtemos:

$$V_{out} = V_{cc} \frac{R_{10k\Omega}}{R_{NTC} + R_{10k\Omega}}. \quad (5)$$

Rearranjando a mesma relação, obtemos a resistência do termistor a partir da tensão medida:

$$R_{NTC} = R_{10k\Omega} \frac{V_{cc} - V_{out}}{V_{out}}. \quad (6)$$

Estas duas expressões, Equações (5) e (6), permitem ligar diretamente a leitura analógica do Arduino à temperatura por meio da equação empírica do NTC (Equação 4).

Prepare o Experimento

1. Lógica do Código: O código tem como objetivo medir a temperatura utilizando um sensor NTC 10k e exibir o valor correspondente no monitor serial do Arduino. Para isso, vamos seguir os seguintes passos:
 - Leitura do sensor: O Arduino lê a tensão V_{out} no ponto intermediário do divisor de tensão, através de uma entrada analógica.
 - Cálculo da resistência do sensor NTC: Com base na tensão medida, o código calcula a resistência do sensor.
 - Conversão de resistência para temperatura: A resistência obtida é convertida em temperatura utilizando a Equação 4.
 - Exibição no monitor serial: A temperatura calculada é impressa.
 - O processo mantém-se em ciclo.
2. Elaboração do código por blocos: o Tinkercad não possui um sensor NTC no banco de elementos. Por este motivo, utilizaremos uma resistência (cerca de 20 k Ω) para simular o referido sensor. Caso necessário, altere o valor da resistência dinamicamente, de forma a simular o sensor. Montagem do circuito:
 - Resistor de 20k Ω (simulando o sensor NTC):
 - Um terminal conectado ao 5V do Arduino.
 - O outro terminal conectado ao ponto central do divisor de tensão e ao pino A0 do Arduino.
 - Resistor de 10k Ω (divisor de tensão):

- Um lado conectado ao ponto central do divisor de tensão (ligado ao resistor de $20k\Omega$ e ao pino A0).
- O outro lado conectado ao GND do Arduino.

3. Montagem do Software por Blocos:

- Selecionar um bloco de leitura analógica e configurá-lo para o pino A0, onde a tensão do divisor de tensão será lida.
- Criar um bloco de operação matemática para converter a leitura do pino A0 em tensão (V_{out}) utilizando a Equação ???. Atenção: o Tinkercad pode se limitar aos números inteiros e reais. Caso isso aconteça, ao copiar o código para a IDE do Arduino, utilize 1023.0 e 5.0. Com este procedimento, iremos garantir que o sistema utiliza números reais.
- Adicionar um bloco de operação matemática para calcular a resistência/sensor NTC com base na leitura de V_{out} . Utilize a equação do divisor de tensão para obter a resistência R_{NTC} (Equação ??). Atenção: o Tinkercad pode definir as variáveis como números inteiros *int*. Para contornar este problema, na IDE do Arduino, altere as definições de *int* para *float*. Não se esqueça de garantir que todos os números das operações matemáticas estão representados com uma casa decimal - caso contrário, serão tratados como um inteiro.
- Criar um bloco de operação matemática para calcular a temperatura com base na resistência R_{NTC} obtida. Implementar a equação de conversão de temperatura (Equação 4) utilizando os coeficientes do sensor. Para esta fase, podemos usar valores típicos: $T_0 = 298$ K, $R_0 = 10 k\Omega$ e $B \approx 3950$ K.
- Criar um bloco de impressão no monitor serial. Configurar para exibir a temperatura (em Kelvin) e a resistência/sensor NTC. Dica: em seguida, na IDE do Arduino você pode editar este bloco. Os comandos:

```
Serial.print(...);
```

```
Serial.println(...);
```

permitem escrever o conteúdo *sem* ou *com* linebreak, respectivamente. Ainda, você também pode adicionar texto ao colocá-lo entre aspas duplas.
- Inserir um bloco de espera antes de repetir o próximo ciclo de leitura.
- Garantir que todas as operações sejam repetidas continuamente, mantendo a medição ativa.
- Este código por blocos não funcionará plenamente, devido as limitações da ferramenta. Por este motivo, edições serão necessárias na IDE do Arduino. Estas edições foram indicadas acima.

Realize o Experimento

1. Calibração do sensor NTC:

- Medir a resistência do sensor NTC utilizando um multímetro em três temperaturas de referência:
 - Água com gelo ($T = 273$ K).
 - Água fervente ($T = 373$ K).

- Temperatura ambiente (confirmada com um termômetro de laboratório).
- Montar uma tabela temperatura (T) *vs.* resistência (R_{NTC}).
- Utilizar o programa **MagicPlot** para ajustar a Equação 4 aos dados obtidos.
- Determinar os parâmetros T_0 , B e R_0 para o sensor NTC utilizado.

2. Montagem do circuito na protoboard:

- Seguir o esquema elétrico do Tinkercad, garantindo a correta ligação do divisor de tensão.
- Transferir o código para o IDE do Arduino, fazendo as adaptações necessárias. Por exemplo:
 - Definir corretamente as variáveis e constantes (valores de T_0 , B , R_0 obtidos na calibração).
 - Ajustar o uso de pontos flutuantes (`float`).
 - Garantir que as leituras e cálculos sigam o modelo correto.
- Testar a leitura do sensor NTC no Arduino e comparar os valores obtidos com o termômetro externo.

Análise os Resultados

- **Análise do Funcionamento:** Descreva como o sistema respondeu às diferentes temperaturas. A leitura variou de forma coerente com o esperado? Houve estabilidade nos valores exibidos? O tempo de resposta foi adequado?
- **Precisão e Calibração:** Compare os valores medidos com o termômetro de referência. A equação utilizada forneceu resultados próximos da realidade? Comente sobre a precisão obtida e se seria necessário refinar a calibração.
- **Mapeamento e Conversão:** Avalie se os parâmetros utilizados na equação (valores de T_0 , B e R_0) foram adequados para seu sensor. Houve algum problema na conversão de resistência para temperatura? A saída do monitor serial foi compreensível?
- **Extensões e Variações:** Proponha variações no sistema, como:
 - Substituir o monitor serial por uma saída visual, como um display LCD.
 - Adicionar alarmes (LEDs ou buzzer) para temperaturas fora de uma faixa ideal.
 - Implementar um sistema de controle térmico automático (ex: ligar um ventilador).
- **Prototipagem Livre:** Com base nos componentes já estudados (LEDs, buzzer, LDR, NTC, potenciômetro, servo motor), proponha um projeto funcional. Exemplos:
 - Estação meteorológica simples com medição de temperatura e luminosidade.
 - Sistema de climatização controlado por temperatura.

- Termômetro digital com alarme para temperaturas críticas.

Descreva o funcionamento do seu projeto no relatório.

Prática Projeto Final 1: Medindo a aceleração da gravidade g por queda livre

Objetivo

Construir um dispositivo autônomo que registre, em um cartão micro SD, a posição de um corpo em queda livre captada por um sensor ultrassônico. A partir do conjunto (*tempo*, *posição*) os alunos ajustarão as equações do movimento uniformemente acelerado para extrair a aceleração da gravidade g .

Materiais

- 1 Placa Arduino Uno com *Sensor Shield*
- 1 Sensor ultrassônico HC-SR04
- 1 Leitor de cartão micro SD (interface SPI) + cartão micro SD
- 1 Botão tátil
- 1 LED + resistor de $220\ \Omega$
- 1 Fonte portátil 5 V
- 1 Case rígida para eletrônica
- 1 Barra ou trilho vertical para guiar a queda livre
- Material de amortecimento (areia, espuma, EVA)

Montagem do Hardware

Todos os componentes (Arduino, shield, leitor micro SD, sensor ultrassônico, botão e LED) devem ficar dentro de uma case rígida, com o sensor voltado para baixo no eixo de queda. O botão inicia e encerra a gravação, enquanto o LED aceso sinaliza que os dados estão sendo captados. A alimentação vem de um power-bank conectado ao Arduino. A case desliza sem rotação por uma barra vertical. Ao fim do percurso, coloque areia ou espuma para amortecer o impacto e proteger a eletrônica.

Software

Desenvolva o esboço do programa em Tinkercad utilizando blocos. Há limitações neste procedimento, como, por exemplo, as bibliotecas de controle do cartão micro SD. Copie, entretanto, o código gerado para o Arduino IDE a fim de incluir as bibliotecas de micro SD e concluir detalhes não suportados no simulador. Os alunos deverão explorar tópicos além do curso – sobretudo a gravação de arquivos CSV no cartão micro SD. Esta etapa deve ser explorada pelos alunos. O fluxo principal é:

1. Setup: inicializar o cartão e criar `queda.csv`; configurar sensor, botão e LED.
2. Loop: enquanto o botão não for pressionado novamente, ler distância a cada Δt (p.ex. 10 ms) e gravar (*tempo*, *posição*) no arquivo. Ao finalizar, fechar o arquivo e desligar o LED.

Procedimento Experimental

Com o sensor calibrado em posições conhecidas, fixe a case no topo da guia, alinhe-a e pressione o botão para iniciar a gravação (LED aceso). Libere a case para cair livremente. Os dados serão coletados até o impacto com o amortecimento. Pressione o botão novamente para encerrar a aquisição, retire o cartão micro SD e copie `queda.csv` para o computador.

Teoria, Ajuste e Conclusão

Adote movimento unidimensional com aceleração constante g , desprezando arrasto:

$$\begin{aligned}v(t) &= v_0 + g t, \\y(t) &= y_0 + v_0 t + \frac{1}{2} g t^2, \\v^2 &= v_0^2 + 2g (y - y_0).\end{aligned}$$

Importe o CSV no MagicPlot, plote y versus t e ajuste a parábola $y = at^2 + bt + c$. O coeficiente quadrático fornece $g = 2a$. Compare com $g_{\text{local}} \approx 9,81 \text{ m s}^{-2}$. Discuta as possíveis fontes de discrepância entre os valores.

Prática Projeto Final 2: Medindo a constante de mola k

Objetivo

Determinar a constante elástica k de uma mola vertical por dois métodos independentes: (i) o alongamento estático provocado pelo peso de uma massa conhecida e (ii) a frequência natural das oscilações livres, registrada com sensor ultrassônico e armazenada em cartão micro SD.

Materiais

- 1 Placa Arduino Uno com *Sensor Shield*
- 1 Sensor ultrassônico HC-SR04
- 1 Leitor de cartão micro SD (interface SPI) + cartão micro SD
- 1 Botão tátil
- 1 LED + resistor de $220\ \Omega$
- 1 Fonte portátil 5 V
- 1 Case rígida para eletrônica
- 1 Barra ou trilho vertical para guiar a queda livre
- Material de amortecimento (areia, espuma, EVA)
- 1 Mola helicoidal ou dinamômetro;
- Conjunto de massas (50 g – 200 g) com gancho;

Montagem do Hardware

Todos os componentes (Arduino, shield, leitor micro SD, sensor ultrassônico, botão e LED) devem ficar dentro de uma case rígida, com o sensor voltado para baixo. O botão inicia e encerra a gravação, enquanto o LED aceso sinaliza que os dados estão sendo captados. A alimentação vem de um power-bank conectado ao Arduino. A case desliza sem rotação por uma barra vertical.

Software

Desenvolva o esboço do programa em Tinkercad utilizando blocos. Há limitações neste procedimento, como, por exemplo, as bibliotecas de controle do cartão micro SD. Copie, entretanto, o código gerado para o Arduino IDE a fim de incluir as bibliotecas de micro SD e concluir detalhes não suportados no simulador. Os alunos deverão explorar tópicos além do curso – sobretudo a gravação de arquivos CSV no cartão micro SD. Esta etapa deve ser explorada pelos alunos. O fluxo principal é:

1. Setup: inicializar o cartão e criar `massaMola.csv`; configurar sensor, botão e LED.

2. Loop: enquanto o botão não for pressionado novamente, ler distância a cada Δt (p. ex. 10 ms) e gravar (*tempo, posição*) no arquivo. Ao finalizar, fechar o arquivo e desligar o LED.

Procedimento Experimental

Com o sistema em repouso, registre o alongamento estático $\Delta y = y_{\text{eq}} - L_0$. Pressione o botão (LED aceso), abaixe a massa alguns centímetros e solte-a, permitindo oscilações livres. Meça as oscilações sem permitir que sejam amortecidas. Pressione o botão novamente para parar a aquisição e transfira o arquivo para o computador.

Teoria e Derivação

Tomando o eixo y positivo para baixo, as forças são:

$$F_g = +mg, \quad F_k = -k(y - L_0).$$

A aplicação da segunda lei resulta em:

$$m\ddot{y} = mg - k(y - L_0).$$

A partir do equilíbrio estático, $\ddot{y} = 0$, obtém-se:

$$y_{\text{eq}} = L_0 + \frac{mg}{k}.$$

Definindo o deslocamento em torno do equilíbrio, temos:

$$x(t) = y(t) - y_{\text{eq}},$$

e a equação de movimento torna-se:

$$m\ddot{x} + kx = 0, \quad \omega_0 = \sqrt{\frac{k}{m}}.$$

A solução harmônica simples é dada por:

$$x(t) = A \cos(\omega_0 t + \varphi), \tag{7}$$

com período

$$T = \frac{2\pi}{\omega_0}.$$

Análise de Dados

(a) Método estático Calcule k por

$$k = \frac{mg}{y_{\text{eq}} - L_0}.$$

(b) Método dinâmico Ajuste $x(t)$ com a Equação 7 e obtenha ω_0 . Determine

$$k = m\omega_0^2.$$

Compare os dois valores de k e discuta eventuais discrepâncias (erros de medição, pequenas perdas de energia, etc).